# Grind

The AI that learns how you actually work — then rewrites your week before you fail.

### RECOMMENDED POSITIONING

Grind is not a productivity tracker and not a prettier to-do list. It is a personal academic operator. It learns your real effort patterns, predicts deadline failure, and intervenes before the collapse happens.

### SIGNATURE FEATURES

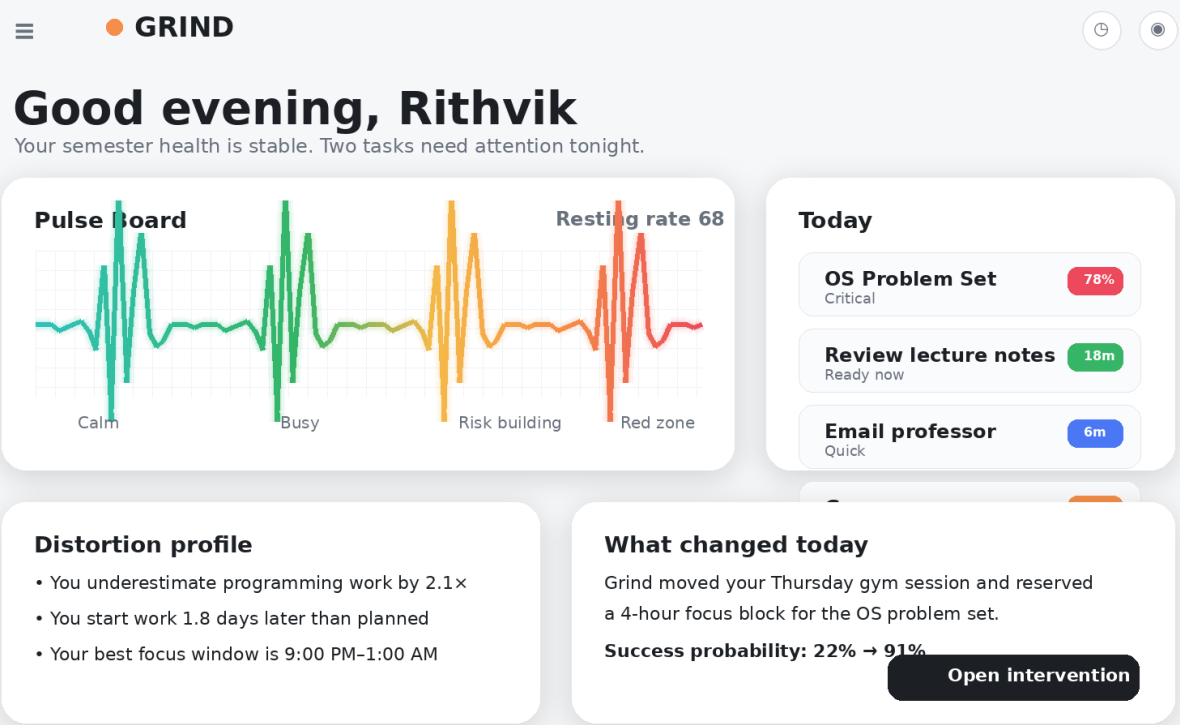
• Distortion Profile — learns where your estimates are wrong
• Failure Predictor — scores each deadline by miss probability
• Pulse Board — heartbeat view of semester health
• Calendar Intervention — automatically creates survivable schedules

### ONE-LINE PITCH

**An AI planner that models the student you really are — not the student you wish you were.**

### THREE REASONS THIS WORKS IN A HACKATHON

1. The demo is concrete in under 20 seconds.
2. The ML story is legible and not just an LLM wrapper.
3. The interface is distinctive without being gimmicky.




Concept view: the Pulse Board dashboard shows semester health, high-risk tasks, the personal distortion profile, and Grind’s latest intervention.

# 1. Product flow and design philosophy

## What Grind must do in 24 hours

The product only needs one believable end-to-end loop: ingest work, estimate it using the user’s distortion profile, detect failure risk, intervene in the calendar, and show the result clearly on the Pulse Board.

| Step | What happens | Why it matters |
|---|---|---|
| 1. Ingest | Assignments arrive from LMS/manual entry and become structured tasks. | Makes the system feel live, not hand-entered. |
| 2. Calibrate | Grind corrects time estimates using the user’s distortion profile. | This is the core novelty. |
| 3. Predict | Failure Predictor flags assignments likely to slip. | Creates urgency and clear value. |
| 4. Intervene | The scheduler adds or moves blocks in Google Calendar. | This is the automation payoff. |
| 5. Launch | Start Mode turns the scary task into the smallest next step. | Removes activation friction. |
| 6. Reflect | Pulse Board and daily recap show semester health and what changed. | Makes the whole system understandable. |

### Minimal, not sterile

Use a calm neutral canvas, one orange accent, and the single hero visual.

### One feature per p

Each screen should answ What is risky? What char right now?

### Explain interventions

Never silently move time. Every resche screen must show before/after logic an new success probability.

## Brand image and language

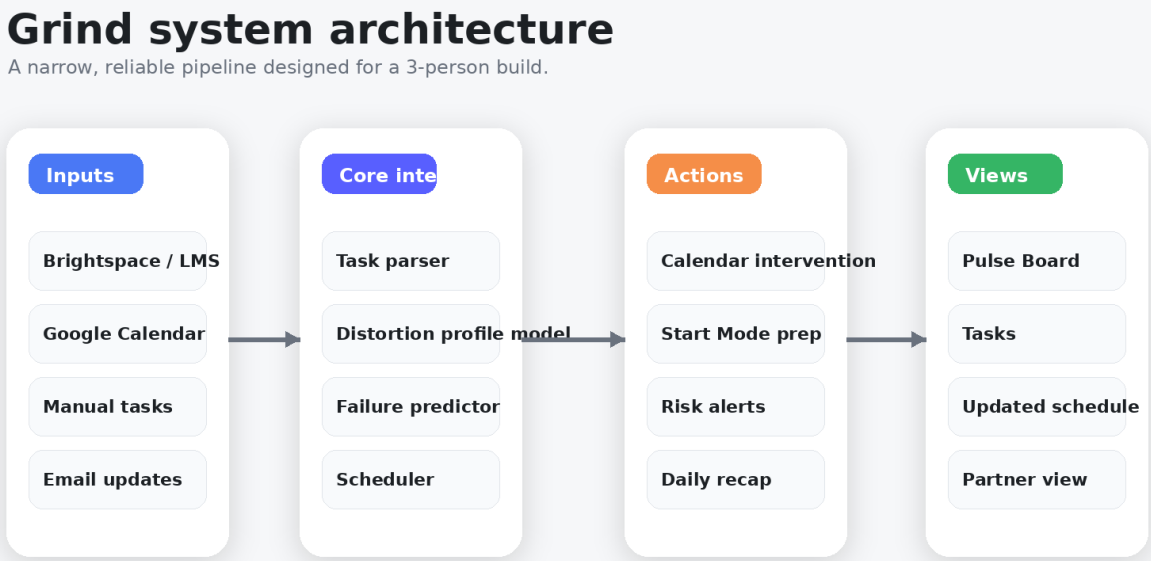
Brand personality: precise, calm, slightly intense, trustworthy. This should feel like an operator’s console for a semester under pressure.

Good pitch language: “personal distortion profile,” “failure predictor,” “calendar intervention,” “semester health,” “start mode,” and “pulse board.”

# 2. Architecture and technical plan

## Recommended stack

| Layer | Choice | Reason |
|---|---|---|
| Frontend | Next.js + TypeScript + Tailwind | Fast UI iteration and polished app surfaces. |
| Database / auth | Supabase | Straightforward auth, Postgres, and fast setup. |
| ML / intelligence | FastAPI + scikit-learn / XGBoost + LLM API | Real predictor plus fast parsing. |
| Calendar | Google Calendar API | The intervention has to be real. |
| LMS ingestion | Brightspace scrape or seeded import | One narrow source is enough for the demo. |
| Background jobs | Simple worker / cron | Needed for recalculation and nightly recaps. |
| Visualization | SVG / Canvas pulse board in React | Fast, crisp, and controllable. |




Suggested architecture: narrow inputs, one intelligence core, real actions, and a very small set of polished views.

# 3. 24-hour workflow for a team of three

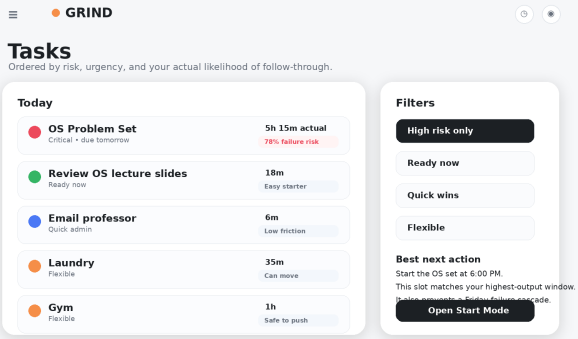
Keep the pipeline narrow. One class-heavy student and one believable intervention are enough for a winning demo. Train a distortion model for corrected effort and a deadline-risk model for miss probability.

| Hours | Focus | Primary owner | Concrete outcome |
|---|---|---|---|
| 0–4 | Project shell, schema, auth, repo discipline | All | App runs end to end with seeded user data. |
| 4–8 | Task ingestion + parser + seeded assignment importer | Builder A | Tasks exist as structured objects with titles, due dates, and estimates. |
| 4–8 | Pulse Board frame + dashboard shell + design system | Builder B | Main dashboard skeleton is visible and polished. |
| 4–8 | Data logging sheet + feature design + starter model notebook | Builder C | Distortion profile training data and feature set are ready. |
| 8–12 | Google Calendar integration + reschedule action | Builder A | One intervention can create or move real events. |
| 8–12 | Task pages, urgent-update page, updated-schedule page | Builder B | Key surfaces are demo-ready. |
| 8–12 | Train first distortion model + generate corrected estimates | Builder C | The core novelty is working with real outputs. |
| 12–16 | Start Mode + smallest-next-step generation | Builder A | The scary task can be launched into action. |
| 12–16 | Finish visual polish, pulse line animation, detail states | Builder B | UI looks product-quality, not hackathon-rough. |
| 12–16 | Train deadline-risk model + expose risk score API | Builder C | Tasks can be ranked by miss probability. |
| 16–20 | Wire models into dashboard + intervention flow | All | A risky task triggers a believable schedule change. |
| 20–24 | Seed polished data, write script, rehearse demo, tighten copy | All | Stable narrative and reliable demo path. |

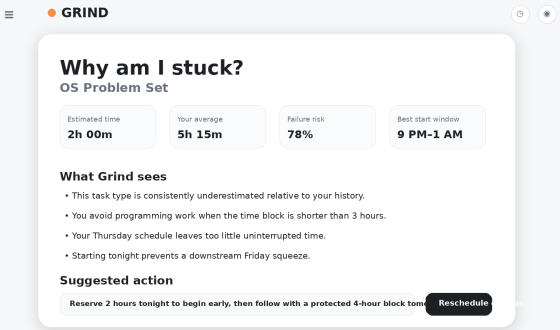
Rule: One assignment going from risky to survivable beats ten thin features. Make the OS-problem-set rescue flow work beautifully.

# 4. UI concept pages

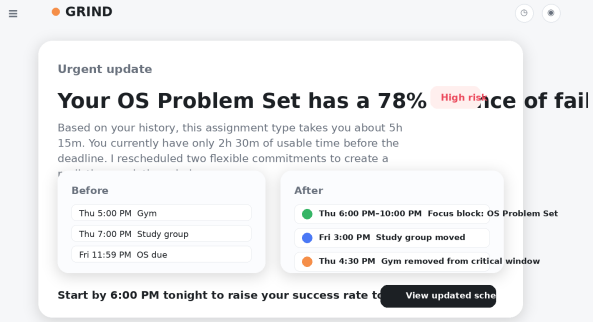
Reference visuals for the build. Keep the UI minimal, coherent, and Apple-like without becoming sterile.



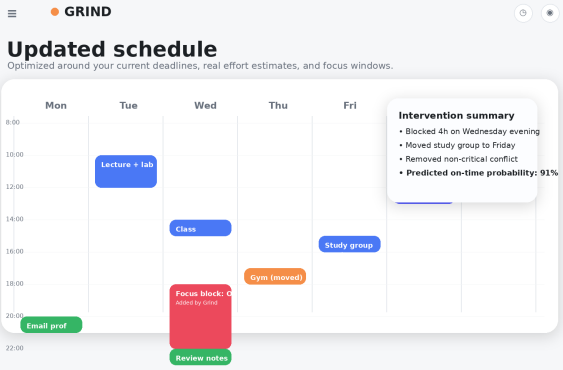

Tasks page — a simple ranked list with risk, real effort, and one best next action.



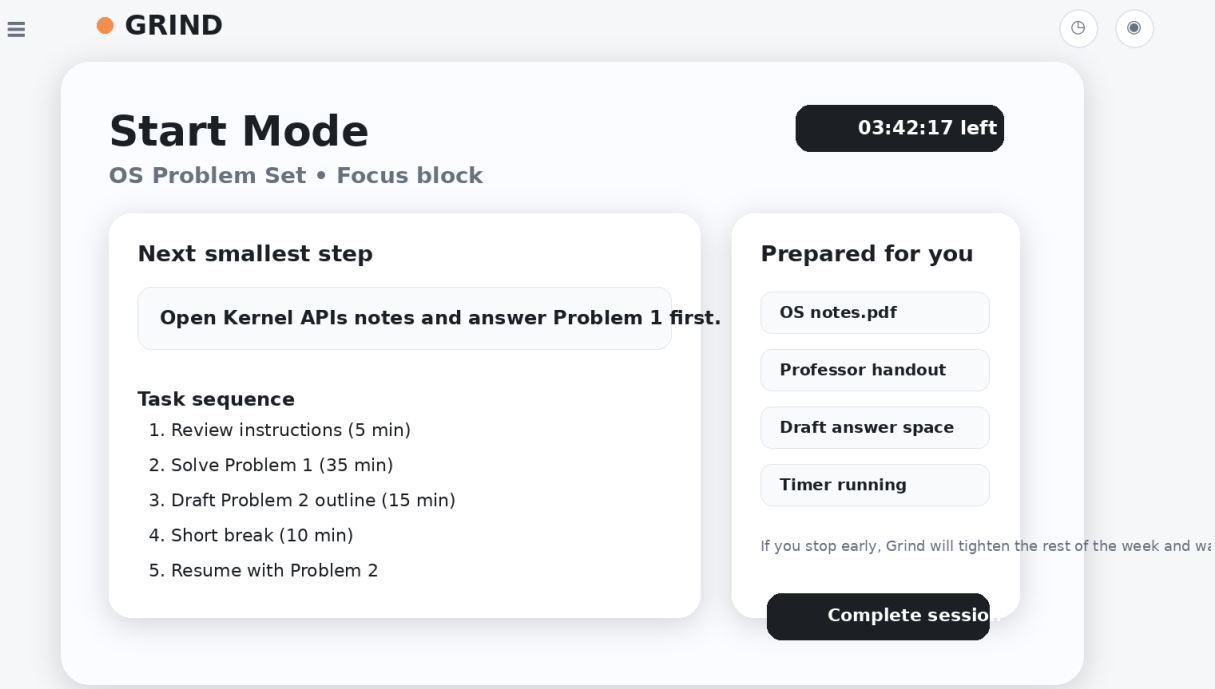

Analysis page — explains why the user is stuck and what Grind is changing.




Intervention page — the strongest single demo screen. It shows the failure risk and the exact schedule changes.




Updated schedule — proves the automation actually did something useful.




Start Mode — turns a vague, scary task into the smallest next step and prepares the workspace.

# 5. How to make this a winning project

## What judges need to understand quickly

Grind is impressive when the judges can say, in plain language: “This system knows when a student is going to fail, and it changes the schedule before that happens.” Keep that line in mind for every product and demo decision.

## Winning checklist

- Use one real-seeming student profile and one emotionally obvious example: a looming problem set, a crowded calendar, and a believable rescue.
- Show the corrected estimate next to the user’s original estimate. That instantly explains the distortion profile.
- Make the intervention screen airtight. Before/after schedule changes are the proof that the automation matters.
- Keep the Pulse Board as the hero visual, but let the AI engine do the hard work.
- Seed excellent data. Bad sample data will make the whole product feel fake.

## Suggested 90-second pitch structure

| | |
|---|---|
| 0–20 sec | “Students do not fail because they forgot a deadline. They fail because they lie to themselves about time, start too late, and build impossible weeks.” |
| 20–45 sec | “Grind learns your personal distortion profile, predicts deadline failure, and rewrites your week before that failure happens.” |
| 45–70 sec | Show the OS problem set at 78% risk, corrected estimate, and the intervention that moves the week from 22% to 91%. |
| 70–90 sec | Close on: “Every productivity app tells you what to do. Grind plans around the person you actually are.” |

## Final recommendation

Build Grind as a calm, serious product. Use strong visuals, but let the core mechanic carry the story: personal effort correction, deadline-risk prediction, and automatic intervention. If those three things work, the project will feel much larger than the time you spent building it.